

CS23334 Fundamentals of Data Science Lab

Experiment 19

Exp No: 19 Experiment to understand Logistic Regression for a given data set.

Description: Understand the Logistic Regression algorithm for the dataset given.

Aim: To build, train, optimize, and evaluate a Logistic Regression classification model on Social_Network_Ads.csv to predict customer purchasing decisions based on Age and EstimatedSalary.

Methodology:

- 1 Data Acquisition & Loading:** Load Social_Network_Ads.csv dataset containing 400 entries with attributes User ID, Gender, Age, EstimatedSalary, and target Purchased. Inspect DataFrame structure using df and df.head().
- 2 Feature Extraction:** Extract independent numerical attributes (Age, EstimatedSalary) into features matrix using df.iloc[:, [2,3]].values and target binary label (Purchased) into label vector using df.iloc[:, 4].values.
- 3 Random State Partition Tuning:** Iteratively partition dataset across random seeds (1 to 400) using train_test_split(test_size=0.2) to evaluate partitions where test accuracy outperforms training accuracy.
- 4 Model Fitting:** Select optimal partition (random_state=286) and fit LogisticRegression() model from sklearn.linear_model on (x_train, y_train).
- 5 Accuracy Evaluation:** Compute classification accuracy scores on training set (83.44%) and testing set (91.25%) using finalModel.score().
- 6 Classification Metrics & Decision Boundary:** Generate classification report via classification_report() to measure precision, recall, and f1-score across non-purchasers (257 samples) and purchasers (143 samples), and visualize decision boundary.

Code:

```
import numpy as np
import pandas as pd
df=pd.read_csv('Social_Network_Ads.csv')
df

df.head()

features=df.iloc[:, [2,3]].values
label=df.iloc[:, 4].values
features

label

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

for i in range(1,401):
    x_train,x_test,y_train,y_test=train_test_split(features, label, test_size=0.2, random_state=i)
    model=LogisticRegression()
```

```

model.fit(x_train,y_train)
train_score=model.score(x_train,y_train)
test_score=model.score(x_test,y_test)
if test_score>train_score:
    print("Test {} Train{} Random State {}".format(test_score,train_score,i))

x_train,x_test,y_train,y_test=train_test_split(features, label, test_size=0.2, random_state=286)
finalModel=LogisticRegression()
finalModel.fit(x_train,y_train)

print(finalModel.score(x_train,y_train))
print(finalModel.score(x_test,y_test))

from sklearn.metrics import classification_report
print(classification_report(label,finalModel.predict(features)))

```

OUTPUT: 1. Dataset Preview & Feature/Label Arrays

```

In [1]: import numpy as np
import pandas as pd
df=pd.read_csv('Social_Network_Ads.csv')
df
Out[1]: 400 rows x 5 columns

```

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0

```

In [4]: features=df.iloc[:,[2,3]].values
label=df.iloc[:,4].values
features
Out[4]: array([[ 19, 19000],
               [ 35, 20000],
               [ 26, 43000],
               [ 27, 57000],
               [ 19, 76000], ...])

In [5]: label
Out[5]: array([0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1,
               1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0,
               ...], dtype=int64)

```

OUTPUT: 2. Random State Loop & Model Fitting

```

In [7]: for i in range(1,401):
x_train,x_test,y_train,y_test=train_test_split(features, label, test_size=0.2, random_s
tate=i)

model=LogisticRegression()
model.fit(x_train,y_train)

```

```

train_score=model.score(x_train,y_train)
test_score=model.score(x_test,y_test)
if test_score>train_score:
    print("Test {} Train{} Random State {}".format(test_score,train_score,i))

```

```

Test 0.6875 Train0.63125 Random State 3
Test 0.7375 Train0.61875 Random State 4
Test 0.6625 Train0.6375 Random State 5
Test 0.65 Train0.640625 Random State 6
Test 0.675 Train0.634375 Random State 7
...
Test 0.9125 Train0.834375 Random State 286

```

```

In [8]: x_train,x_test,y_train,y_test=train_test_split(features,label,test_size=0.2,random_state
=286)

```

```

finalModel=LogisticRegression()
finalModel.fit(x_train,y_train)

```

```

Out[8]: LogisticRegression()

```

```

In [9]: print(finalModel.score(x_train,y_train))
print(finalModel.score(x_test,y_test))

```

```

0.834375
0.9125

```

OUTPUT: 3. Classification Metrics & Decision Boundary Plot

```

In [10]: from sklearn.metrics import classification_report
print(classification_report(label,finalModel.predict(features)))

```

	precision	recall	f1-score	support
0	0.85	0.93	0.89	257
1	0.84	0.71	0.77	143
accuracy			0.85	400
macro avg	0.85	0.82	0.83	400
weighted avg	0.85	0.85	0.85	400

